

UniversidadeVigo

ESCOLA SUPERIOR DE ENXEÑARÍA INFORMÁTICA

Memoria do Traballo de Fin de Grao que presenta

D^a. Tatiana María Quintas Rodríguez

para a obtención do Título de Graduado en Enxeñaría Informática

**Desarrollo y evaluación de modelos de aprendizaje automático
para la predicción de la biodiversidad del suelo**



Septembro, 2026

Traballo de Fin de Grao N^o: 120

Titor/a: Javier Rodeiro Iglesias

Cotitor/a: Antonio Adrián González Pedrouzo

Área de coñecemento: Linguaxes y Sistemas Informáticos

Departamento: Informática

Dedicatoria

Agradecimientos

Índice

1. Introducción	7
2. Objetivos	8
3. Antecedentes y contexto	9
4. Resumen de la solución propuesta	11
5. Marco teórico o práctico	13
5.1. Fundamentos del aprendizaje automático	13
5.1.1. Tipos de aprendizaje automático	14
5.1.1.1. Aprendizaje supervisado	14
5.1.1.2. Aprendizaje no supervisado	14
5.1.1.3. Aprendizaje semi-supervisado	14
5.1.1.4. Aprendizaje por refuerzo	14
5.2. Herramientas empleadas	14
5.2.1. Lenguaje de programación, control de versiones y librerías	14
5.2.2. Servicios de obtención de datos remotos	14
5.2.3. Programas de redacción y documentación	14
5.2.4. Programas de desarrollo de código	14
5.2.5. Programas de creación de diagramas e ilustraciones	14
5.3. Arquitectura general	15
5.3.1. Capa de ingesta de datos	16
5.3.2. Capa de procesamiento de datos	17
5.3.3. Capa de modelado predictivo	18
5.3.4. Capa de evaluación de modelos	21
5.4. Modelos de aprendizaje automático empleados	23
5.4.1. Regresión Ridge	23
5.4.2. Random Forest	23
5.4.2.1. Random Forest multisalida	23
5.4.3. XGBoost	23
5.4.3.1. XGBoost multisalida	23
5.4.4. Redes neuronales	23
5.4.4.1. MLP multisalida	23
5.4.4.2. MLP con función de pérdida personalizada	23
5.4.5. Comparativa de modelos	23
5.5. Selección de variables	23
5.5.1. Métodos de filtrado	23
5.5.2. Métodos de envoltura	23
5.5.3. Métodos de embebido	23
6. Planificación y seguimiento	24
6.1. Planificación	24
6.2. Puntos críticos	25
6.2.1. Desarrollo y parametrización de los modelos predictivos	26
6.2.2. Evaluación de modelos y control de calidad (QA)	26
6.2.3. Estrategias de mitigación	26
6.3. Seguimiento	27
6.4. Justificación de desviaciones	27
7. Principales aportaciones	28
8. Conclusiones	29
9. Vías de trabajo futuro	30

10. Referencias	31
11. Anexos	34
11.1. Glosario de Términos	34

Índice de imágenes

Figura 1 «Diagrama con las diferentes fases de CRISP-ML(Q)»	11
Figura 2 Arquitectura de software del proyecto	15
Figura 3 «Diagrama de Gantt con la planificación del TFG.»	25

Índice de tablas

Tabla 1	Tabla de los datasets resultantes de la fase 1	18
Tabla 2	Tabla de submódulos empleados de la librería scikit-learn	19
Tabla 3	Tabla de submódulos empleados de la librería PyTorch	20
Tabla 4	Tabla de submódulos empleados de la librería scikit-learn (para evaluación)	22
Tabla 5	Planificación del TFG.	24
Tabla 6	Distribución de horas y porcentajes de esfuerzo estimados	25
Tabla 7	Glosario de términos y acrónimos	34

1. Introducción

El suelo alberga aproximadamente el 59% de la biodiversidad de nuestro planeta. Este conjunto de organismos genera un gran impacto en el ciclo de nutrientes, el mantenimiento de las estructuras del suelo y la salud de los ecosistemas. En última instancia, la vida que encontramos en los suelos representa la base funcional de los ecosistemas terrestres^[1].

Por estos motivos, resulta fundamental poder controlar y monitorizar el estado de los suelos y su biodiversidad. Sin embargo, nos encontramos con numerosos desafíos a la hora de medir dicha biodiversidad, entre los cuales destacamos los siguientes:

1. El número elevado de especies.
2. Las dificultades metodológicas según el grupo de especies estudiado.
3. La falta de procesos estandarizados.
4. El elevado coste de los diversos análisis a realizar^[2].

Debido a dichos desafíos y limitaciones, surge la necesidad de desarrollar nuevas técnicas que permitan y faciliten el conocimiento de los niveles de biodiversidad en el suelo, asegurando también la fiabilidad y la calidad de los resultados.

En el marco del proyecto europeo SOB4ES («Integrating SOil Biodiversity to Ecosystem Services: testing cost-effectiveness of Soil Biodiversity indicators and the provision of soil biodiversity-based Ecosystem Services to build better land management solutions that effectively implement the EU Soil Strategy»^[3]), en el que se enmarca el presente TFG, se han obtenido mediciones de múltiples propiedades físicas, químicas y ambientales de más de 400 puntos geográficos diferentes a lo largo del continente europeo y sus proximidades. Adicionalmente, se ha evaluado la biodiversidad de dichos suelos a través de múltiples características.

2. Objetivos

El objetivo del presente trabajo es realizar un estudio de los datos del proyecto para desarrollar, entrenar y evaluar modelos de aprendizaje automático que sean capaces de aproximar los valores de biodiversidad para nuevos puntos geográficos, a partir de sus propiedades físicas, químicas y ambientales.

A partir de este objetivo central se proponen los siguientes objetivos específicos:

1. Analizar e integrar los datos provenientes tanto por parte del proyecto europeo SOB4ES con datos climáticos, satelitales y edafológicas externas procedentes de bases de datos geoespaciales y diversos servicios API europeos como Google Earth Engine (GEE) y Copernicus (CDS).
2. Preparar y armonizar los datos disponibles mediante un proceso reproducible de limpieza, normalización y homogeneización de las distintas fuentes heterogéneas, obteniendo un conjunto de datos consistente que sirva de base fiable para el análisis y modelado posterior.
3. Seleccionar y justificar, desde un punto de vista metodológico, las técnicas y herramientas de análisis de datos y aprendizaje automático más adecuadas a las características del problema (tamaño muestral reducido, múltiples indicadores correlacionados entre sí, datos heterogéneos procedentes de fuentes distintas).
4. Entrenar, comparar y evaluar métricamente diferentes algoritmos de aprendizaje automático supervisado para aislar y determinar la solución algorítmica más precisa, robusta y con mayor capacidad de generalización.
5. Garantizar el rigor metodológico y la reproducibilidad de los resultados obtenidos, mediante la aplicación sistemática de una metodología de gestión de proyectos de ciencia de datos (CRISP-ML(Q)) que permita mitigar riesgos y sesgos a lo largo del proceso de investigación.

3. Antecedentes y contexto

El proyecto europeo SOB4ES, en cuyo marco se desarrolla este TFG, surge como respuesta a la necesidad de la Unión Europea de contar con indicadores fiables y de bajo coste sobre la biodiversidad del suelo, en contexto de la Estrategia de Suelos de la UE (EU Soil Strategy). Tradicionalmente, la evaluación de la biodiversidad edáfica se ha basado en muestreos de campo y análisis de laboratorio, que si bien son precisos, presentan las limitaciones ya descritas en la Introducción: coste elevado, falta de estandarización metodológica entre grupos y especies y dificultad para escalar todo a nivel continental o internacional.

Ante estas limitaciones, en los últimos años se han extendido el uso de modelos de aprendizaje automático supervisado para estimar indicaciones ambientales y de biodiversidad a partir de variables predictoras más económicas de obtener (datos climáticos, satelitales y edafológicos), reduciendo así la dependencia de mediciones directa exhaustivas sobre el terreno.

Varios trabajos han abordado directamente el modelado de biodiversidad del suelo haciendo uso del aprendizaje automático, con resultados que nos pueden servir de referencia para contrastar con los resultados obtenidos en este TFG (véase **Sección 8**):

1. Salako et al.^[4] emplearon **Random Forest** para modelar la distribución espacial de la riqueza y densidad (ind./m²) de tres grupos ecológicos de lombrices (epigeas, endogeas y anécicas) en Alemania, usando como predictores variables de suelo y clima a distintos usos del terreno.

Encontraron que cada grupo ecológico responde a predictores distintos: las especies epigeas están mayoritariamente condicionadas por el clima en zonas forestales, mientras que las endogeas, el grupo más diverso, dependen sobre todo de la textura del suelo (contenido de arcilla y limo).

2. Un estudio de modelado conjunto multiespecie^[5] entrenó una **red neuronal profunda multi-tarea** para predecir la distribución de 77 especies de lombrices en Francia a partir de variables climáticas, edáficas y de cobertura del terreno, empleando SHAP para identificar los principales factores ambientales. El modelo conjunto alcanzó un TSS ≥ 0.7 y mejoró notablemente las predicciones para especies raras frente a los modelos de distribución de especies tradicionales (de una sola especie).

3. Un estudio sobre biodiversidad multi-trófica del suelo mediante metabarcoding de ADN ambiental ^[6] comparó sistemáticamente **LightGBM, Random Forest y una red neuronal (ANN)** para predecir la abundancia relativa de 51 grupos tróficos (bacterias, hongos, protistas, oligoquetos, insectos, colémbolos y otros metazoos), usando cuatro configuraciones de variables predictoras: datos ambientales de baja resolución, datos in situ de alta calidad, embeddings de imágenes satelitales, y una combinación de ambos.

Random Forest obtuvo consistentemente el mejor rendimiento, ligeramente por encima de LightGBM, y los datos in situ de alta calidad superaron sistemáticamente tanto a los datos de baja resolución como a los derivados de imágenes satelitales. Un resultado directamente relevante para este TFG, que también combina variables in situ (SOB4ES) con variables remotas de menor resolución (Google Earth Engine, Copernicus).

4. Un estudio de predicción del microbioma del suelo^[7] comparó seis modelos de aprendizaje automático clásico (incluyendo Random Forest y Gradient Boosting) frente a un modelo de aprendizaje profundo para predecir la frecuencia relativa de comunidades bacterianas y fúngicas a partir de factores ambientales.

Gradient Boosting obtuvo el mejor resultado a nivel de filo ($R^2 = 0.57$), y Random Forest y Gradient Boosting empataron como mejores modelos a nivel de grupo funcional ($R^2 = 0.45$); las arquitecturas más sofisticadas de aprendizaje profundo (CNN, transformers) obtuvieron resultados peores, atribuido explícitamente al tamaño reducido de los datos de entrenamiento disponibles. Esto es una conclusión especialmente relevante para este TFG, cuyo conjunto de datos (400 muestras) es de un orden de magnitud similar.

En conjunto, estos antecedentes apuntan de forma consistente a que los modelos basados en árboles, como Random Forest, Gradient Boosting y XGBoost, pueden igualar o hasta superar a los modelos basados en redes neuronales cuando el conjunto de datos, especialmente en el caso de los datos relacionados con la biodiversidad (que son reducidos) y a que los datos in situ de alta calidad siguen siendo, por ahora, más informativos que los derivados exclusivamente de fuentes remotas con resoluciones bajas. Estas dos hipótesis pueden ser contrastadas empíricamente con los resultados de esta TFG.

Este enfoque indirecto no es exclusivo de la biodiversidad del suelo. El uso de variables climáticas, satelitales y edáficas como predictoras de propiedades del ecosistema está ya consolidado en ámbitos afines, de los cuales se resaltan los siguientes:

- Estimación del carbono orgánico en el suelo.
- Clasificación de usos del terreno.
- Predicción del rendimiento agrícola a partir de series temporales de NDVI.

La disponibilidad reciente de catálogos satelitales abiertos de alta resolución y de reanálisis climáticos globales, ha reducido significativamente la barrera de entrada para poder aplicar estas técnicas a nuevos problemas ecológicos, entre los cuales se encuentra el que aborda en este TFG.

Frente a estos antecedentes, la aportación diferencial que puede proporcionar este TFG es doble: por un lado, la integración de múltiples fuentes de orígenes heterogéneos en un mismo pipeline reproducible y versionado; por otro lado la comparación de múltiples algoritmos de aprendizaje supervisado bajo un mismo marco de validación, en lugar de limitarse a un único modelo, lo que permite discutir con evidencia empírica cuál generaliza mejor ante un conjunto de datos comparativamente reducido.

Este TFG se sitúa bajo el contexto siguiente: parte de los datos previamente recolectados por el proyecto SOB4ES en más de 400 puntos geográficos europeos, y los complementa con fuentes de datos remotas de acceso abierto, con el objetivo de evaluar hasta qué punto los modelos de aprendizaje automático supervisado pueden aproximar la biodiversidad del suelo a partir de variables indirectas, relativamente más baratas y escalables que las obtenidas mediante el muestreo directo.

4. Resumen de la solución propuesta

La solución propuesta se basa en un enfoque integral de ciencia de datos, diseñado para abarcar desde la recopilación de múltiples fuentes de información hasta la evaluación de modelos predictivos.

Para estructurar este proceso, se ha adoptado la metodología de desarrollo CRISP-ML(Q)[3], que permitirá definir y realizar el trabajo de manera estructurada. Además de la metodología CRISP-ML(Q), a lo largo del desarrollo de este trabajo se aplicará en conjunto una metodología de desarrollo iterativa incremental [4].

CRISP-ML(Q) (Cross-Industry Standard Process of Machine Learning with Quality Assurance) es el estándar de facto para la gestión del ciclo de vida en proyectos de aprendizaje automático. A diferencia de la ingeniería de software tradicional o la minería de datos clásica (CRISP-DM[23]), el desarrollo de modelos predictivos exige una aproximación principalmente experimental, no lineal e iterativa, además de ser un proyecto en el cual el rendimiento de los modelos puede verse degradado con el tiempo.

Para dotar a este proceso de un marco de ingeniería robusto y estricto, CRISP-ML(Q) organiza el flujo de trabajo en seis fases cíclicas interconectada entre sí, integrando así, de forma transversal, los diversos controles de QA (Quality Assurance) en cada una de las transiciones.

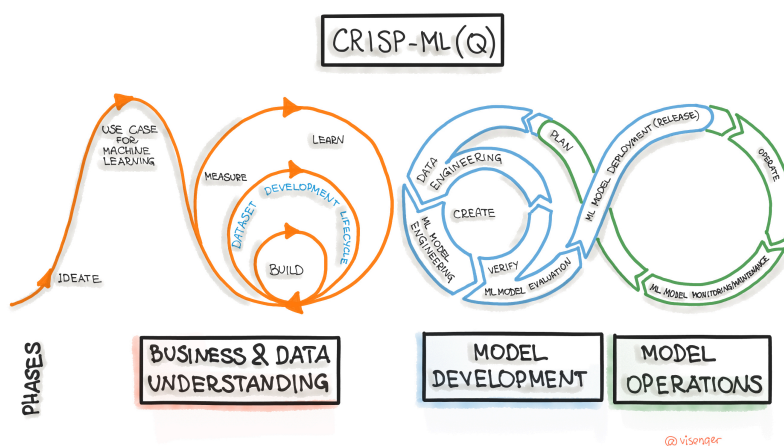


Figura 1: «Diagrama con las diferentes fases de CRISP-ML(Q)»

En el diagrama anterior, podemos distinguir las siguientes fases:

1. **Comprensión del negocio y de los datos:** Definición del problema predictivo y auditoría inicial de la viabilidad de las fuentes proporcionadas.
2. **Preparación de los datos:** Fase de ingeniería de característica y de datos, limpia, unificación y estandarización de los datos de las fuentes base. Junto también con la obtención de datos externos.
3. **Ingeniería del modelo:** Selección de algoritmos/tipos de modelo, entrenamiento formal, ajuste de hiperparámetros y su consecuente desarrollo de los modelos necesarios.
4. **Evaluación del modelo:** Validación metrológica, es decir con los correspondientes parámetros de evaluación, del rendimiento estadístico frente a datos de control ciegos (nunca empleados en entrenamiento ni en otras pruebas anteriores a la evaluación).
5. **Despliegue:** Entrega o integración del modelo o pipeline reproducible en el entorno operativo.
6. **Monitoreo y mantenimiento:** Control continuo para mitigar la degradación de las predicciones en el tiempo y aplicar las correspondientes medidas de mantenimiento.

Además, tal como se indica anteriormente con CRISP-ML(Q), al ser incremental, también se aplican técnicas de desarrollo iterativo incremental.

El **desarrollo iterativo incremental** se basa en la capacidad de poder dividir un proyecto en diversos bloques reducidos y asignarlos a bloques temporales generalmente fijos, los cuales pueden ser adaptados a futuro. Estos bloques reducidos se denominan como **iteraciones**.

De forma más específica las iteraciones se pueden definir como miniproyecto o como un set de tareas en las cuales “*en todas ellas se repite un proceso de trabajo similar para proporcionar un resultado completo sobre el producto final*[9]”. Cada iteración tiene como resultado una parte del producto final funcional, el cual se va integrando a lo largo de las iteraciones hasta crear el producto final.

Todos los detalles relacionados con las fases, hitos y los tiempos de realización de todo el proyecto se pueden encontrar en la siguiente sección.

La aplicación de ambas metodologías nos permite lo siguiente: **1. Adquisición e Integración de Datos:** Para alimentar los modelos, no solo se utilizarán los datos propios del proyecto SOB4ES (disponibles en archivos y carpetas como EARTHWORMS_RAW/) , sino que se complementarán y completarán con bases de datos y mapas europeos de resolución variable.

Las fuentes externas empleadas son las siguientes:

- Bases de datos geoespaciales como la *European Soil Database* (ESDAC) y *CORINE Land Cover* para clasificar el tipo de suelo, uso de la tierra, propiedades físicas (arena, arcilla, densidad) y propiedades químicas (pH, metales pesados, carbono orgánico) de las parcelas.
- Uso de APIs como *Google Earth Engine* (earthengine-api) para extraer medias de temperatura diaria, humedad relativa e índices de vegetación (NDVI).
- Uso de *Copernicus Climate Data Store* (cdsapi) para registrar precipitaciones mensuales acumuladas y modelos de elevación digital (DEM).

2. Procesamiento de la Información:

El tratamiento masivo de estos datos ambientales se realizará utilizando diversas bibliotecas de Python.

Se utilizarán pandas, numpy y múltiples otras librerías para la limpieza, manipulación eficiente de variables tabulares y escritura de archivos CSV/XLSX. Para el manejo de datos geoespaciales y archivos multidimensionales provistos por Copernicus y bases europeas, se implementarán herramientas específicas como rasterio, xarray y dbfread.

3. Modelado y Evaluación:

Una vez consolidado el conjunto de datos definitivo, la solución final consistirá en el diseño, entrenamiento e implementación de diferentes algoritmos de aprendizaje automático, aplicando en cada uno de los diferentes modelos la metodología explicada previamente y cuyas fases/etapas se encuentran en las secciones de [Planificación y seguimiento] y Arquitectura.

Estos algoritmos mapearán las complejas relaciones entre las variables ambientales y la biodiversidad del suelo.

Finalmente, los modelos serán evaluados y comparados métricamente para aislar la solución algorítmica más robusta y generalizable para futuras predicciones geográficas, siguiendo el ciclo de evaluación-mejora descrito en la fase de **Ingeniería del modelo y Evaluación del modelo** de CRISP-ML(Q).

5. Marco teórico o práctico

En esta sección se describe el marco teórico y práctico en el que se apoya y fundamenta nuestras conclusiones: la arquitectura del pipeline de datos y modelado, las tecnologías y herramientas de tercero empleadas, como también los fundamentos teóricos que nos permite crear, desarrollar, entender e interpretar los resultados que nos proporcionan los diferentes modelos seleccionados.

5.1. Fundamentos del aprendizaje automático

El término de aprendizaje automático, también conocido como Machine Learning (ML) fue acuñado por primera vez por el informático Arthur Samuel[32][31] en 1959, conocido por su trabajo en la implementación de un ajedrez que aprendiera de forma autónoma y por dar soporte y colaborar en lo que hoy se conoce como LaTeX[34].

Él acuñó el término de ML por primera vez en el artículo “Some Studies in Machine Learning Using the Game of Checkers” publicado en el IBM Journal of research and development[33], definiéndolo de la siguiente forma:

“Machine Learning es el campo de estudio que permite le otorga a las computadoras la capacidad de aprender sin explícitamente programarlo.”

Otro personaje relevante dentro del campo de la inteligencia artificial es Tom Mitchell[35][31], creador del ordenador que es capaz de “leer la mente humana” a partir de imágenes del cerebro humano[36]. Él lo define de una forma más técnica, siendo esta una de las definiciones más citadas dentro de la literatura académica[37], la cual es la siguiente:

“Se dice que un programa de ordenador aprende de una experiencia E con respecto a una tarea T y una medida de rendimiento P, si su rendimiento en T, medido por P, mejora con la experiencia E”

Esta definición es la adoptada como referencia en este trabajo, al descomponer con claridad los tres elementos presentes del proyecto:

- **Tarea (T):** predecir los indicadores de biodiversidad del suelo a partir de las diferentes variables ambientales.
- **Experiencia (E):** El conjunto de datos de campo del proyecto SOB4ES, combinado con variables climáticas y geoespaciales de fuentes remotas.
- **Medida de rendimiento (P):** las métricas descritas en [poner zona], la cual se encuentra un poco más adelante en este documento.

Estas dos definiciones se pueden resumir de la siguiente forma:

“El Machine Learning es una rama o subconjunto de la Inteligencia Artificial que se centra en dotar a los algoritmos la capacidad de “aprender”. Este aprendizaje es a partir de la detección de patrones en los datos de entrenamiento para, posteriormente, hacer predicciones o inferencias sobre nuevos datos.”[40]

Esta capacidad de aprender, permite a los modelos de ML realizar predicciones, tomar decisiones y hacer acciones variadas sin que lo tengan estrictamente indicado dentro de su código. También lo permite diferenciarse de los sistemas expertos tradicionales, en los que el conocimiento se codifica explícitamente, y lo acerca a la estadística de la que toma buena parte de su base matemática.

5.1.1. Tipos de aprendizaje automático

Dentro del aprendizaje automático, existen muchos algoritmos y múltiples formas de aprendizaje en base al tipo de datos o experiencias de las que aprenden.

Dichas formas de aprendizaje se clasifican en cuatro tipos generales, los cuales se explican a continuación.

5.1.1.1. Aprendizaje supervisado

5.1.1.2. Aprendizaje no supervisado

5.1.1.3. Aprendizaje semi-supervisado

5.1.1.4. Aprendizaje por refuerzo

5.2. Herramientas empleadas

5.2.1. Lenguaje de programación, control de versiones y librerías

5.2.2. Servicios de obtención de datos remotos

5.2.3. Programas de redacción y documentación

5.2.4. Programas de desarrollo de código

5.2.5. Programas de creación de diagramas e ilustraciones

5.3. Arquitectura general

Para proporcionar una respuesta a los objetivos planteados y permitir la predicción de la biodiversidad del suelo a partir de variables ambientales, se ha diseñado una arquitectura de software basada en un flujo de datos modular y desacoplado, permitiendo de esta forma que cualquier cambio en alguna de las capas no provoque tener que realizar cambios en el resto de las capas.

El uso de este enfoque garantiza la reproducibilidad del sistema ante la posibilidad de incluir nuevas fuentes de datos o nuevos algoritmos de aprendizaje automático.

La arquitectura explicada previamente se compone de cuatro capas, las cuales transforman los datos brutos en uno o varios modelos predictivos para su futura evaluación y comprobación. Dicha estructura de capas se puede ver en el siguiente diagrama.

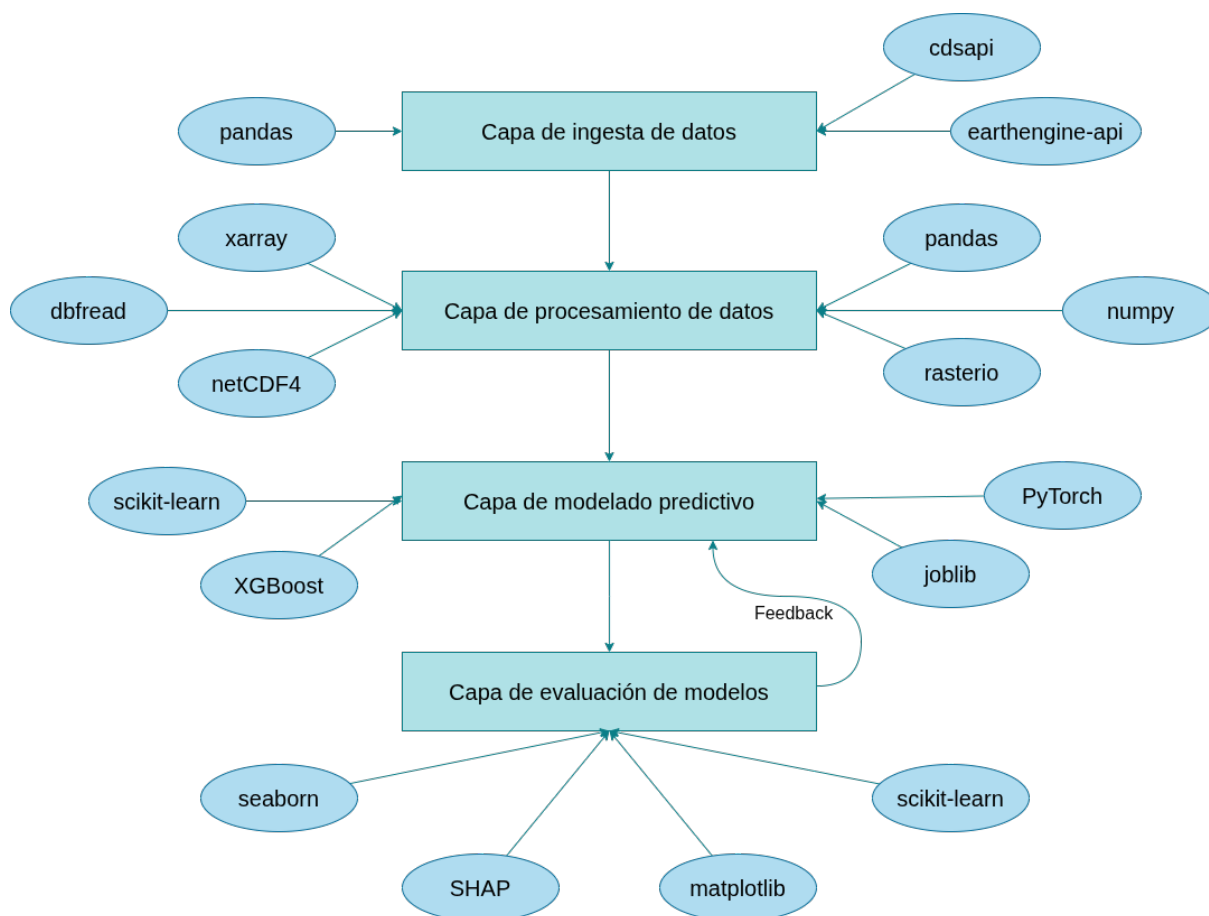


Figura 2: Arquitectura de software del proyecto

5.3.1. Capa de ingesta de datos

Esta capa se encarga de la adquisición de datos con las diferentes APIs a emplear, como también cargar los diferentes archivos de los datasets que se tengan en local.

La adquisición de datos se realiza en dos fases, las cuales están basadas en el origen de los datos que se van a recolectar:

1. Ingesta de datos locales:

- Se hace la lectura de los datos locales recolectados por el grupo europeo SOB4ES, como otras fuentes de datos Europeas almacenados en local, los cuales contienen las capas raster de la ESDAC, CORINE Land Cover, entre otras fuentes de datos que nos proporcionan información fisicoquímica de las zonas en las que se hicieron las diferentes pruebas.

2. Ingesta de datos remotos (APIs):

- Se implementa mediante el uso de APIs para obtener otros datos no recogidos dentro de los datos locales como viene siendo datos climatológicos y otros de interés como la inclinación del terreno.
- Para ello se hacen uso de los siguientes servicios:
 1. **Google Earth Engine (GEE):** A través del módulo earthengine-api de python, para la extracción automatizada de datos como las imágenes satelitales de Sentinel-2 usadas para el cálculo de índices de vegetación y medias de temperatura y humedad.[4]
 2. **Copernicus Climate Data Store (CDS):** Mediante el módulo cdsapi de python para la descarga de datos de reanálisis climático, en los que se incluyen datos sobre las precipitaciones mensuales acumuladas.[5]
 3. **Copernicus DEM (vía AWS):** Descarga del DEM (Digital Elevation Model), empleado para la obtención de altitudes, pendientes y orientaciones del terreno.[6]

Además de las fases de la adquisición de datos, destacamos las siguientes librerías empleadas para ello:

- **pandas:** Librería empleada para la manipulación e ingeniería de datos tabulares. Permite la lectura y la escritura de archivos .csv y .xlsx (excel). También puede gestionar uniones, filtrados y operaciones vectorizadas sobre múltiples filas de forma no muy compleja y eficaz.
- **earthengine-api:** Librería de desarrollo oficial de Google que permite la conexión remota, la autenticación y la orquestación de scripts de procesamiento geoespacial sobre la API de Google Earth Engine.
- **cdsapi:** Librería que habilita un enlace directo y oficial a la API de CCDS, para el envío de solicitudes de extracción de datos parametrizadas y para la descarga automática de los datos meteorológicos solicitados.

Al final de todo el procesamiento de esta capa se crea un archivo requirements.txt cuyo objetivo es almacenar todas las librerías necesarias para que lo realizado pueda ser reproducible.

Nota: A la hora de realizar la ingesta de datos remotos, tomamos como referencia las coordenadas obtenidas a través de la ingesta de datos locales. Esto permite la obtención de los datos necesarios de forma eficiente y evitar obtener datos innecesarios o crear muchos más datos faltantes.

5.3.2. Capa de procesamiento de datos

Esta capa tiene como objetivo unificar las diferentes fuentes de datos bajo un marco de coordenadas común y resolver las diferentes discrepancias entre los datos, estandarizándolos y armonizándolos para poder emplearlos en futuras capas.

Este proceso se hace dividiéndolo en tres subprocesos o fases:

1. **Procesado de datos de ficheros:** En este subproceso nos centramos en los datos que se tienen ya tanto del proyecto SOB4ES, como de otros datasets ya existentes, tanto a nivel UVIGO, como a nivel europeo. Este procesamiento de datos da como resultado varios elementos, que será usado en el tercer subproceso. Dichos elementos son los siguientes:
 1. **Datasets:**
 - **model_ready.csv:** Dataset optimizado ya para su uso e introducción en los diferentes modelos que se desarrollen a futuro.
 - **clean.csv:** Dataset intermedio con todos los datos limpios, empleado en las siguientes fases del procesamiento de datos como referencia y para la armonización de sus datos con los datos obtenidos por consultas remotas a bases de datos en la red.
 - **imputation_flags.csv:** Dataset que contiene únicamente flags que indican en qué celdas faltan datos de cada tipo de dato introducido en clean.csv.
 2. **scaler.pkl:** Archivo pickle que guarda la media y la desviación estándar calculadas en la preparación de los datos y usadas para la normalización de cualquier otro dato nuevo de forma idéntica a como se normalizaron los datos originales.
 3. **label_encoders.pkl:** Archivo pickle que guarda el mapeo de categorías a números aprendido durante la preparación de datos, para codificar de la misma forma cualquier otro dato nuevo.
2. **Procesado de datos de consultas remotas a bases de datos:** En este subproceso nos centramos en los datos obtenidos a partir de la realización de solicitudes en remoto para obtener datasets o datos provenientes de los diferentes servicios web que se emplean dentro de este trabajo. Como consecuencia, este procesamiento de datos da como resultado un segundo dataset y un scaler.pkl intermedios con todos los datos relevantes obtenidos de datasets online, que será usado en el tercer subproceso.
3. **Combinación y armonización de ambas fuentes:** En este último subproceso, hacemos uso de los datasets intermedios comentados previamente para combinarlos y armonizar sus datos. Como resultado de este último subproceso, obtendremos un dataset final con todos los datos preparados para ser usados en los diferentes modelos que se desarrollen en la siguiente capa.

Además de los subprocesos, a lo largo de procesamiento de esta datos se hace uso de las siguientes librerías:

- **pandas:** véase el punto anterior.
- **numpy:** Librería especializada en computación numérica que es generalmente empleada para el manejo de datos numéricos complejos, así como para dar soporte matemático a otras librerías como pandas.
- **rasterio:** Herramienta geoespacial destinada a la lectura de archivos matriciales con formato GeoTIFF, procedentes de las diferentes fuentes de datos europeas. También sirven para la realización de consultas directamente a las URLs correspondientes al DEM de Copernicus.
- **xarray:** Librería diseñada para la manipulación de conjuntos de datos multidimensionales, los cuales se encuentran distribuidos en múltiples archivos .nc (NetCDF).
- **dbfread:** Librería ligera de bajo nivel capaz de leer de forma nativa y eficiente archivos con formato .bdf, facilitando la extracción de la información tabular contenida en ciertos conjuntos de datos vectoriales del catálogo europeo.
- **netCDF4:** Librería que permite la lectura, escritura y manipulación de archivos con formatos netCDF y HDF5.

5.3.3. Capa de modelado predictivo

El objetivo principal de esta capa es el desarrollo de los diferentes modelos predictivos como también la preparación de los sets de aprendizaje y pruebas y su consecuente entrenamiento.

Antes de comenzar a trabajar en esta capa es recomendable tener el dataset con los datos finales preparado y armonizado para que el proceso pueda, de esta forma, ser mucho más lineal y sencillo de seguir.

Esta capa y la siguiente, van a estar comunicadas entre sí debido a que el feedback que se reciba de la **Capa de evaluación de modelos** va a ser usada para mejorar los modelos que se desarrollen en esta capa. De esta forma se pueden obtener múltiples modelos predictivos de mayor calidad, el cual es el quinto sub objetivo de este trabajo de fin de grado.

De la misma forma que las capas anteriores, esta capa va a estar dividida en varias fases o subprocesos:

1. **Fase de partición de datos:** Esta fase consiste en coger el dataset final con los datos ya normalizados y armonizarlos y dividirlos en tres sets de menor tamaño para emplearlos cada uno para una de las siguientes funciones.
 1. **Set de Entrenamiento:** Conjunto inicial, el cual será empleado para que los modelos que se desarrollen aprendan los patrones que haya ocultos entre los datos mediante el ajuste de los parámetros de los modelos.
 2. **Set de Validación:** Conjunto usado para el ajuste de los hiperparámetros de los modelos y seleccionar cuáles funcionan mejor de forma objetiva sin contaminar en proceso. En esta parte también se hará uso del feedback obtenido de la Capa de evaluación de modelos.
 3. **Set para la evaluación:** Conjunto de datos no empleados en ninguno de los sets mencionados previamente, su uso es para evaluar la eficacia del modelo en condiciones reales y con datos con los cuales nunca ha trabajado.

La división de los datos se ha hecho teniendo en cuenta el origen de las muestras tomadas y a partir de la realización de esta fase obtenemos los datasets indicados en la siguiente tabla:

Nombre CSV	Porcentaje de datos	Nº de entradas
train.csv	70%	299
test.csv	15%	64
eval.csv	15%	65

Tabla 1: Tabla de los datasets resultantes de la fase 1

2. **Fase de investigación de posibles modelos predictivos:** El objetivo de esta fase es tener una lista de modelos predictivos que se puedan desarrollar y emplear teniendo en cuenta los requisitos de este trabajo, como también condicionantes como el formato de datos y el número de estos mismos.

También en esta fase, además del listado de modelos predictivos a desarrollar, se elaborará un listado de parámetros a utilizar como variables para que el modelo tenga ya cierta calidad a la hora de hacer el primer aprendizaje y evaluación.
3. **Fase de configuración del entorno:** El objetivo de esta fase es tener el entorno con las librerías y todos los componentes necesarios para poder ejecutar sin problemas los diferentes modelos que se desarrollen a lo largo de la siguiente fase.

El resultado de esta fase es una versión actualizada del archivo requirements.txt con todas las librerías empleadas en esta capa.

4. **Fase de desarrollo de modelos:** El objetivo de esta fase es tener un par de modelos desarrollados y contenidos dentro de múltiples jupyter notebooks para poder así, de esa forma, se pueden efectuar dentro de un entorno cerrado y seguro, además de poder ser subidos y ejecutados online en plataformas como Google Collab. El resultado de esta fase son múltiples notebooks con cada uno de los modelos desarrollados para su futuro entrenamiento.
5. **Fase de entrenamiento de los modelos:** El objetivo de esta fase es una vez creados los diferentes modelos a emplear, estos serán ejecutados uno a uno y serán entrenados para, posteriormente evaluarlos y realizar las correcciones correspondientes. El resultado de la fase es múltiples archivos **.pkl** con todos los modelos entrenados para su futuro uso.

A lo largo del desarrollo de esta capa, se hacen uso de las siguientes librerías:

- **sklearn/scikit-learn:** Librería principalmente creada para machine learning y para análisis de datos. Dentro de la propia librería hacemos uso de las funciones/submódulos que se muestran en la siguiente tabla:

Submódulo	Descripción
linear_model.Ridge	Implementa la regresión lineal con regularización L2, siendo este el algoritmo base para Ridge Regression.
ensemble.RandomForest	Implementación del algoritmo de Random Forest para regresión.
multioutput. Regressor-Chain	Permite envolver un modelo base (en nuestro caso Random Forest) para, durante el entrenamiento, replicar el modelo base una vez por cada target. De esta forma se pueden encadenar las predicciones de los targets anteriores como entrada adicional para poder predecir los siguientes valores.
model_selection. RandomizedSearchCV	Realiza la búsqueda de hiperparámetros, probando todas las posibles combinaciones del grid de parámetros establecido por el usuario/desarrollador. Se escoge el mejor set de hiperparámetros haciendo uso de la validación cruzada (CV).
model_selection. cross_validate	Ejecuta la validación cruzada repetida para medir la robustez del modelo y poder detectar sobreajustes.
model_selection. RepeatKfold	Permite aplicar la estrategia de partición en pliegues para la validación, obteniendo así lo siguiente: $x \text{ pliegues} * y \text{ repeticiones} = xy \text{ evaluaciones}$
model_selection.KFold	Variante simple de RepeatKfold .

Tabla 2: Tabla de submódulos empleados de la librería scikit-learn

- **XGBoost:** Librería que implementa algoritmos de ML bajo el framework de Gradient Boosting[]. Su aprendizaje se basa en árboles de decisión potenciados por el framework mencionado previamente. Además de hacer uso de la propia librería destacamos el siguiente submódulo o función:
 - **XGBRegressor:** Implementa el algoritmo de gradient boosting sobre los árboles de decisión. Además, de que permite trabajar con múltiples salidas haciendo uso del argumento `multi_strategy=multi_output_tree`.

- **PyTorch:** Librería que se emplea principalmente para actividades de ML, y para la creación de redes neuronales. Actualmente es una de las más empleadas dentro del entorno académico e investigador.

De esta librería destacamos los siguientes elementos:

Submódulo/Módulo	Descripción
torch	Librería principal de PyTorch, proporcionando los tensores, el cálculo de gradientes (autograd) y el optimizador Adam usado para entrenar a las redes neuronales.
torch.nn	Módulo de torch encargado de la definición y construcción de redes neuronales. En términos de definición se pueden definir las capas, el modo de activación, la regularización y la función de pérdida, entre otros elementos.
torch.utils.data. DataLoader	Se encargan de gestionar la división del conjunto de entrenamiento en lotes de trabajo (batches) y su iteración aleatoria durante cada período de entrenamiento.
torch.utils.data. TensorDataset	

Tabla 3: Tabla de submódulos empleados de la librería PyTorch

- **joblib:** Librería que permite la serialización de las funciones como también la computación paralela. Esto es de gran utilidad a la hora de entrenar y trabajar con los diferentes modelos, especialmente para manejar la concurrencia y la persistencia de los modelos creados haciendo uso de scikit-learn y XGBoost.

5.3.4. Capa de evaluación de modelos

El objetivo principal de esta capa final es evaluar los diferentes modelos desarrollados en la Capa de modelado predictivo y a partir de las evaluaciones realizadas, hacer las siguientes acciones:

1. Escoger los modelos que presentan una mayor utilidad y funcionalidad para el proyecto.
2. Mejorar los modelos escogidos en múltiples iteraciones.

Una vez más, de forma similar a las capas anteriores, esta capa se divide en las siguientes fases:

1. **Fase de validación individual:** Durante el desarrollo de los modelos, en cada notebook se han integrado varias celdas que tienen como objetivo realizar la validación cruzada para cada modelo antes de generar todos los modelos resultantes.

Esto se realiza para comprobar ya en la construcción de los modelos si los parámetros escogidos son los más efectivos o si estos, por otro lado, están provocando un sobreajuste en el modelo. También permite prevenir la fuga de datos originado por un entrenamiento deficiente o mal preparado.

De esta fase se obtienen datos que figuran en los outputs de los notebooks de cada uno de los modelos diseñados, además de los modelos resultantes que serán usados en la siguiente fase.

2. **Fase de validación en conjunto de los modelos generados:** Para el correcto desarrollo de esta fase se van a crear dos notebooks más que se van a encargar de realizar la evaluación y validación de los modelos con dos puntos de vista distinto, los cuales son los siguientes:

1. **Punto de vista numérico:** En este notebook nos centramos en cargar todos los modelos desarrollados y evaluar el rendimiento de cada uno de los modelos contra el dataset creado para las pruebas.

Dentro de este punto de vista nos centramos en las siguientes métricas, cuyas definiciones se pueden encontrar en el [INSERTAR ANEXO]:

- R2 global.
- R2 por target.
- RMSE.
- MAE.

A partir de la ejecución del notebook se podrán ver los resultados de las pruebas tanto en las celdas de salida del notebook, como en los archivos de resultado que se exportan en la última fase de ejecución. Estos archivos contienen los resultados de la comparación entre modelos y el R2 por cada target de todos los modelos probados.

2. **Punto de vista discreto:** En este notebook sigue una metodología similar de realización de pruebas en comparación con el notebook anterior, pero para este se realiza una clasificación para poder ver los resultados, no tanto como números, sino como aciertos y fallos.

Dentro de este punto de vista nos podemos centrar en otras métricas, cuyas definiciones también se pueden encontrar en el [INSERTAR ANEXO]:

- Matrices de confusión.
- Kappa de Cohen.
- F1-macro.

3. **Análisis de resultados y aplicación de mejoras:** El objetivo principal de esta fase es analizar los resultados obtenidos en la fase anterior, en la primera vuelta escoger los modelos que presentan el mejor funcionamiento y, a partir de la segunda vuelta, aplicar mejoras para intentar mejorar aún más el rendimiento.

Esta fase no requiere de desarrollo de código, es únicamente ver los resultados obtenidos, analizarlos y mejorarlos en la medida de lo posible en base a las capacidades y limitaciones que se tienen.

A lo largo del desarrollo de esta capa, se hacen uso de las siguientes librerías:

- **scikit-learn:** véase definición anterior. Para esta capa hacemos uso de los siguientes submódulos de scikit-learn:

Submódulo	Descripción
metrics.r2_score	Calcula el coeficiente de determinación R ² , la métrica principal para medir qué proporción de la varianza real da explicado el modelo.
metrics.mean_squared_error	Calcula el error cuadrático medio, empleado para medir el error de predicción penalizando más de esa forma los errores grandes.
metric.mean_absolute_error	Calcula el error absoluto medio (MAE).
inspection.permutation_importance	Mide la importancia de cada variable predictora mediante el barajeo de sus valores y observando en cuánto cae el rendimiento del modelo. Se emplea en los modelos los cuales presentan compatibilidad con SHAP (modelos que no son de árbol) y también para complementar a SHAP.
base.BaseEstimator	Clases empleadas para envolver un modelo creado con PyTorch en una interfaz compatible con scikit-learn. Es necesario para poder hacer uso de permutation_importance sobre redes neuronales (son incompatibles de base).
base.RegressorMixin	

Tabla 4: Tabla de submódulos empleados de la librería scikit-learn (para evaluación)

También se hacen uso de los submódulos RepeatedKFold y cross_validate, los cuales ya se han explicado en la parte de librerías empleadas de la Capa de modelado predictivo.

- **SHAP:** Librería que calcula los valores shapley haciendo uso de TreeExplainer para determinar la contribución exacta de cada variable a cada predicción realizada. Esto nos permite ver de forma gráfica cómo funciona el modelo y también ver cuánto impacto tiene cada una de las variables en los diferentes targets establecidos.
- **matplotlib:** Librería creada para la generación de visualizaciones de diferentes tipos en Python, permitiendo mostrar datos de forma gráfica y crear ilustraciones gráficas complejas de forma intuitiva y sencilla. También funciona como motor subyacente para otras librerías como shap y seaborn.
- **seaborn:** Librería basada en matplotlib, permitiendo la creación de gráficas de datos con una interfaz de mayor nivel en comparación con matplotlib y está integrado de forma más cercana con otras librerías empleadas en este trabajo como pandas.

5.4. Modelos de aprendizaje automático empleados

5.4.1. Regresión Ridge

5.4.2. Random Forest

5.4.2.1. Random Forest multisalida

5.4.3. XGBoost

5.4.3.1. XGBoost multisalida

5.4.4. Redes neuronales

5.4.4.1. MLP multisalida

5.4.4.2. MLP con función de pérdida personalizada

5.4.5. Comparativa de modelos

5.5. Selección de variables

5.5.1. Métodos de filtrado

5.5.2. Métodos de envoltura

5.5.3. Métodos de embebido

6. Planificación y seguimiento

6.1. Planificación

La planificación de este trabajo ha empezado a partir del inicio del mismo, en mayo de 2026, a partir de ahí se planifica su desarrollo de la forma en la que se indica en la siguiente tabla:

Período	Fase CRISP-ML(Q)	Actividades Planificadas	Hitos
Mayo	Comprensión y preparación inicial de los datos.	Ingesta, limpieza, armonización e integración de los datos (ficheros y fuentes remotas).	Datasets intermedios de los datos en ficheros como de los datos de solicitudes remotas.
1 jun - 15 jun	Cierre de la preparación de los datos.	Correcciones de los notebooks de ingesta y procesamiento de datos. Búsquedas iniciales sobre modelos predictivos.	Datasets de datos finales. Lista de posibles modelos predictivos a probar.
16 jun - 1 jul	Modelado de modelos	Desarrollo de los modelos para entrenar. Entrenar los diferentes modelos y experimentar con las diferentes variables posibles.	Modelos entrenados. Documentación aparte sobre los diferentes modelos desarrollados.
2 jul - 15 ago	Evaluación de modelos y mejoras	Evaluar los modelos teniendo en cuenta las métricas resultantes de todas las pruebas. Mejorar los diferentes modelos en base a los resultados obtenidos en las pruebas.	Informes sobre las pruebas realizadas en los diferentes modelos y sus correspondientes resultados. Modelos predictivos mejorados (múltiples iteraciones de ellos) en base a lo obtenido en las pruebas anteriores.
16 ago - sept	Documentación final, revisión y entrega	Completar la limpieza y el refinamiento de la documentación final. Comprobar que todo lo elaborado funciona correctamente. Preparar el material de soporte necesario para la defensa.	Documentación revisada y pulida. Materiales de apoyo para la defensa (i.e. presentación, documentación adicional). Todos los archivos requeridos para la entrega.

Tabla 5: Planificación del TFG.

Dentro de la planificación no se tiene en cuenta los elementos relacionados con la documentación, debido a que este es un elemento que se ha planificado realizar a lo largo de toda la duración de este trabajo de fin de grado. Esto permite tener todos datos y todos los progresos realizados ya redactados para facilitar las actividades de pulido y mejora de la documentación de la memoria del TFG.

También como se puede ver en la planificación, la aplicación de CRISP-ML(Q) se ha adaptado a las necesidades de este trabajo, motivo por el cual los nombres de las fases no coinciden completamente con lo indicado en la correspondiente documentación sobre la metodología de desarrollo.

En el siguiente Diagrama de Gantt, podemos ver de forma más gráfica la planificación que se ha realizado para la elaboración de este trabajo.

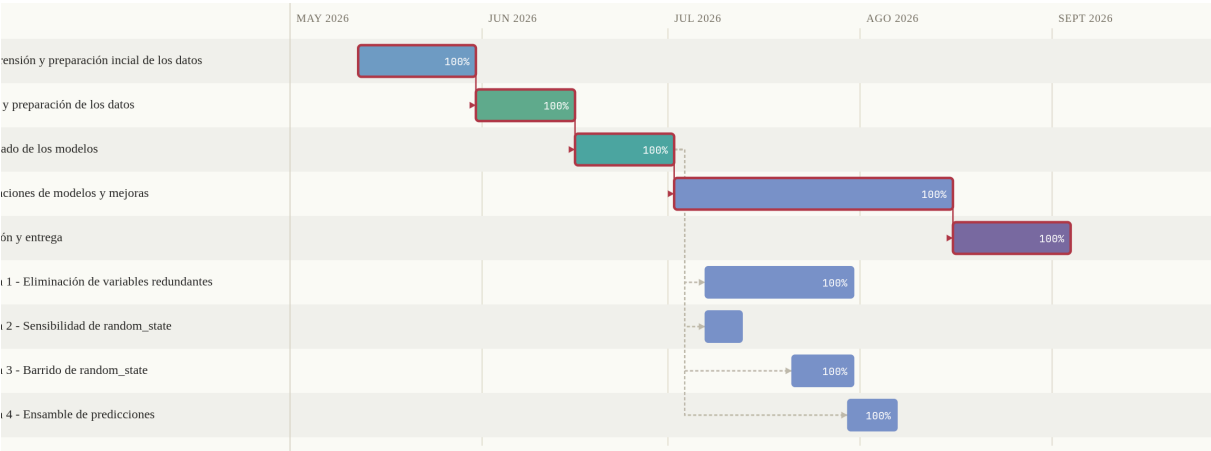


Figura 3: «Diagrama de Gantt con la planificación del TFG.»

Además del diagrama de Gantt mostrado en el diagrama anterior, en la siguiente tabla se muestra la distribución de horas y porcentaje de esfuerzo estimado para cada una de las correspondientes fases de desarrollo de este trabajo.

Fase CRISP-ML(Q)	Duración estimada (en horas)	Porcentaje de esfuerzo estimado (%)
Comprensión y preparación inicial de los datos	50	18.3%
Cierre de la preparación de los datos	40	13.3%
Modelado de los modelos	70	23.3%
Evaluación de modelos y mejoras	110	36.7%
Documentación final, revisión y entrega	30	10%
Total	300	100%

Tabla 6: Distribución de horas y porcentajes de esfuerzo estimados

6.2. Puntos críticos

Dentro de un proyecto, especialmente dentro del ciclo de vida del mismo, los puntos críticos se definen como aquellos componentes, fases o tareas, que debido a la complejidad técnica o al desconocimiento inicial al empezar el proyecto, presentan un riesgo más elevado de poder provocar desviaciones en el cronograma establecido inicialmente. También pueden afectar en mayor o menor medida a la calidad del producto final.

Dentro del marco de este TFG, los principales puntos críticos se concentran en el desarrollo de modelos y la evaluación formal de los mismos. A continuación, se describirán los diferentes puntos críticos y sus riesgos de forma más detallada y las estrategias de mitigación aplicadas.

6.2.1. Desarrollo y parametrización de los modelos predictivos

El desarrollo y parametrización de los modelos predictivos se considera como un punto crítico debido a que el rendimiento final del sistema va a depender de las decisiones tomadas al inicio del desarrollo. Entre esas decisiones se destacan las siguientes:

1. Número y tipo de modelos a entrenar y desarrollar.
2. Selección del espacio y tiempo de búsqueda/cómputo disponible.

La selección de modelos debe de contar con el hecho de que se dispone de un set reducido de datos. Si se seleccionan modelos, los cuales no operan bien con pocos datos, entonces como consecuencia los rendimientos van a ser mucho peores de lo esperable.

Además, aún escogiendo modelos capaces de operar con pocos datos, siempre va a haber cierta cantidad de modelos que presenten sobreajuste ya por defecto. Esto mismo se podrá ver a lo largo del desarrollo de la memoria y en el **Anexo III**.

Un ajuste deficiente de hiperparámetros o una mala elección del espacio de búsqueda inicial puede provocar un sobreajuste mucho mayor al ya predispuesto por el hecho de la falta de datos.

6.2.2. Evaluación de modelos y control de calidad (QA)

La evaluación formal de los modelos es muy importante, porque de ella depende la selección final de los algoritmos a emplear. Un error en esta fase podría invalidar todas las conclusiones de este trabajo.

Además, la calidad de los datos de entrada, condiciona directamente a la fiabilidad de cualquier métrica obtenida, por lo que un fallo no detectado en las capas inferiores del flujo de trabajo puede propagarse silenciosamente hasta la evaluación final.

6.2.3. Estrategias de mitigación

Para reducir, en la medida de lo posible, los riesgos provenientes de los puntos anteriores se han aplicado las siguientes medidas de mitigación:

- Uso sistemático de validación cruzada repetida en todos los modelos desarrollados, en lugar de hacer uso de una única partición de validación, para obtener estimaciones más robustas y precisas sobre el rendimiento real.
- Separación estricta del dataset original en tres subconjuntos (entrenamiento, validación y evaluación) desde la fase inicial de modelos de modelos. Esto permite evitar que el set de evaluación se use en ningún momento en otros procesos para los cuales no fue creado, como puede ser la búsqueda de hiperparámetros.
- Registro de los campos sin datos de los datasets originales mediante un archivo de banderas de imputación. Esto permite tener un mayor conocimiento y control sobre los datos que se poseen o que siguen carentes en todo momento, además, permite la determinación sobre qué datos son mediciones reales y cuáles son estimaciones.
- Revisiones periódicas del proceso de desarrollo en forma de reuniones semanales (véase **Sección 6.3**), permitiendo así detectar errores o desviaciones en el desarrollo y parametrización de los modelos antes de que afecten a fases posteriores.
- Realización de pruebas complementarias para la simplificación y mejor selección de los modelos finales (véase **PONER SECCIÓN**). Dichas pruebas también sirven para medir la robustez actual de los modelos.

6.3. Seguimiento

El seguimiento del progreso del presente TFG se realiza mediante la celebración de reuniones periódicas, generalmente semanales, entre el alumnado, el tutor y el co-tutor del proyecto. Estas reuniones constituyen el mecanismo principal de control y coordinación a lo largo de todo el desarrollo del trabajo.

Estas reuniones tienen los siguientes objetivos:

1. **Comunicación de avances:** exponer el trabajo realizado durante el período transcurrido desde la anterior reunión, incluyendo los resultados obtenidos así como las dificultades o problemas encontrados.
2. **Resolución de dudas:** plantear y aclarar todas las cuestiones técnicas, metodológicas o de alcance que hayan surgido durante el desarrollo de las tareas.
3. **Definición de próximos pasos:** Establecer y acordar las tareas a abordar durante el siguiente intervalo temporal, ajustando la planificación en función del estado real del proyecto.

Este esquema de seguimiento continuo permite mantener una supervisión constante sobre la evolución del proyecto, facilitando la detección temprana de desviaciones respecto a la planificación inicial y posibilitando la adopción de medidas correctoras cuando resulta necesario. Asimismo, la periodicidad semanal proporciona un marco de trabajo estructurado que favorece la organización del esfuerzo en tiempos cortos y bien definidos, contribuyendo a un desarrollo incremental y controlado del trabajo.

En casos en los que las circunstancias del proyecto lo requiera, por ejemplo, antes hitos relevantes o la necesidad de abordar decisiones de mayor importancia, la frecuencia de las reuniones puede ajustarse, intensificando o espaciando su frecuencia en función de las necesidades puntuales del desarrollo.

6.4. Justificación de desviaciones

A fecha de 30/07/2026, no existen diferencias con la planificación creada.

7. Principales aportaciones

Teniendo en cuenta los objetivos que se han establecido al principio de este TFG, los cuales se encuentran en la sección de Objetivos, las principales aportaciones que se proporcionan son las siguientes:

1. **(Objetivos 1 y 2) Evidencia práctica sobre la compatibilidad entre datos in situ y datos remotos de menor resolución para la estimación de biodiversidad del suelo:** la integración y armonización de las mediciones de campo de alta calidad del proyecto SOB4ES con fuentes remotas de acceso abierto (Google Earth Engine, Copernicus) aporta un punto de comparación adicional a la evidencia ya existente sobre si los datos in situ siguen ofreciendo una ventaja informativa frente a los derivados exclusivamente de teledetección (véase Antecedentes y contexto), además de un proceso de armonización documentado que podría reutilizarse con otros grupos de especies o campañas de muestreo del propio proyecto.
2. **(Objetivo 3) Justificación metodológica de qué técnicas de aprendizaje automático son adecuadas para las características concretas de este problema:** la selección de algoritmos (regresión regularizada, ensamblados basados en árboles y redes neuronales) se ha justificado explícitamente en función de las limitaciones del problema, tamaño muestral reducido (400 observaciones) e indicadores de salida potencialmente correlacionados entre sí, en lugar de partir de una elección arbitraria o exhaustiva de algoritmos.
3. **(Objetivo 4) Evidencia empírica sobre qué familia de algoritmos generaliza mejor en la predicción de biodiversidad del suelo con un conjunto de datos reducido:** la comparación sistemática de regresión Ridge, Random Forest, XGBoost, RegressorChain y redes neuronales MLP bajo un mismo marco de validación cruzada repetida y una misma partición estricta de entrenamiento, validación y evaluación permite contrastar empíricamente, con datos propios, la hipótesis recurrente en la literatura revisada de que los modelos basados en árboles igualan o superan a las redes neuronales cuando el conjunto de datos es pequeño.
4. **(Objetivo 4) Evaluación de si el modelado conjunto (multi-salida) aporta ventaja frente al modelado independiente por indicador:** al entrenar variantes multi-salida (rf_multisalida, xgb_multisalida, mlp_multisalida, regressorchain) junto con sus equivalentes de salida única bajo las mismas condiciones, este trabajo aporta evidencia sobre si aprovechar las relaciones entre distintos indicadores de biodiversidad del suelo mejora o no las predicciones frente a tratarlos de forma independiente, una pregunta que la literatura revisada no responde de forma directa para este dominio.

A esto se suma el resultado de la prueba de eliminación de variables sobre la relevancia real de los micronutrientes del suelo: la evaluación sistemática del efecto de eliminar cu_z, ni_z y mo_z, de forma individual y combinada, aporta evidencia sobre si estas variables son realmente necesarias para la predicción o si su contribución es redundante frente al resto de variables climáticas y edáficas empleadas. Dichos resultados se pueden ver con más detalle en el ANEXO (sabedios).

5. **(Objetivo 5) Aplicación de una metodología de gestión de proyectos de ciencia de datos como marco de rigor metodológico:** el uso sistemático de CRISP-ML(Q) a lo largo de este trabajo ha permitido anticipar y mitigar riesgos habituales en estudios de este tipo, como por ejemplo, el sobreajuste con conjuntos de datos reducidos, fuga de información entre las fases de entrenamiento y evaluación, entre otros, mediante decisiones metodológicas concretas (partición estricta de datos, validación cruzada repetida), que quedan documentadas y podrían servir de referencia para futuros trabajos de investigación dentro del propio proyecto SOB4ES.

8. Conclusiones

9. Vías de trabajo futuro

A partir del trabajo realizado, se han identificado las siguientes líneas de continuación:

1. **Ampliar la cobertura geográfica y temporal del conjunto de datos** mediante la incorporación de nuevas campañas de muestreo dentro del proyecto SOB4ES a medida que estén disponibles, para reducir la dependencia del modelo de un conjunto de datos reducido.
2. **Extender la comparación a otros grupos de biodiversidad del suelo** más allá de los ya integrados en los diferentes modelos creados para este trabajo, replicando el mismo pipeline y metodología.
3. **Explorar posibles arquitecturas de aprendizaje profundo adicionales** como, por ejemplo, los modelos basados en embeddings de imágenes satelitales descritos previamente en Antecedentes y contexto, una vez que se dispongan de un mayor volumen de datos que permita mitigar de forma más eficiente el riesgo de sobreajuste ya existente a causa de la falta de datos.
4. **Automatizar el reentrenamiento periódico de los modelos**, cerrando de esta forma la fase de “Monitoreo y mantenimiento” de CRISP-ML(Q), la cual se encuentra descrita en Sección 4, de forma que el sistema pueda incorporar nuevas mediciones sin tener que repetir todo el proceso de forma manual.
5. **Exponer los modelos finales a través de un servicio API** que permita obtener una predicción de biodiversidad para un nuevo punto geográfico a partir de sus variables ambientales, facilitando así el uso por parte de terceros ajenos al desarrollo de este TFG.
6. **Crear un sistema multi-modelo** que permita realizar las predicciones de todos los targets teniendo en cuenta qué modelos funcionan mejor para cada target. Esto permite que todos los targets que se empleen en un futuro puedan ser empleados sin que un modelo en específico los haga inutilizables como target.

10. Referencias

- [1] M. A. Anthony, S. F. Bender, y M. G. A. van der Heijden, «Enumerating soil biodiversity», 2023. [En línea]. Disponible en: <https://doi.org/10.1073/pnas.2304663120>
- [2] FAO, *State of Knowledge of Soil Biodiversity: Status, Challenges and Potentialities*. 2020. [En línea]. Disponible en: <https://doi.org/10.4060/cb1928en>
- [3] SOB4ES, «Integrating Soil Biodiversity into Ecosystem Services». [En línea]. Disponible en: <https://www.sob4es.eu/>
- [4] G. Salako, A. Zaitsev, B. Betancur-Corredor, y D. J. Russell, «Modelling and Spatial Prediction of Earthworms Ecological-Groups Distribution Reveal Their Habitat and Environmental Preferences», *SSRN*, 2024, [En línea]. Disponible en: <https://ssrn.com/abstract=4925999>
- [5] «Digging deeper: deep joint species distribution modeling reveals environmental drivers of Earthworm Communities», *ResearchGate*, 2025, [En línea]. Disponible en: <https://www.researchgate.net/publication/392737042>
- [6] «Limits and promises of Earth Observation Foundation Models in Predicting Multi-Trophic Soil Biodiversity», *bioRxiv*, 2025, [En línea]. Disponible en: <https://www.biorxiv.org/content/10.1101/2025.05.16.654504>
- [7] «Soil microbiome prediction using traditional machine learning and deep learning models», *Scientific Reports*, 2026, [En línea]. Disponible en: <https://www.nature.com/articles/s41598-026-39537-w>
- [8] S. Studer *et al.*, «Towards CRISP-ML(Q): A Machine Learning Process Model with Quality Assurance Methodology», 2021. [En línea]. Disponible en: <https://doi.org/10.3390/make3020020>
- [9] Codificando Bits, *Los sets de entrenamiento, validación y prueba*, (16 de junio de 2023). [En línea Video]. Disponible en: <https://www.youtube.com/watch?v=79K93XBOsIg>
- [10] Na y Na, «Sets de Entrenamiento, Test y Validación», *Aprende Machine Learning*. [En línea]. Disponible en: <https://www.aprendemachinelearning.com/sets-de-entrenamiento-test-validacion-cruzada/>
- [11] «ML-ops.org». [En línea]. Disponible en: <https://ml-ops.org/content/crisp-ml>
- [12] «Google Earth Engine». [En línea]. Disponible en: <https://earthengine.google.com/>
- [13] «Homepage | Copernicus». [En línea]. Disponible en: <https://www.copernicus.eu/en>
- [14] «NumPy». [En línea]. Disponible en: <https://numpy.org/es/>
- [15] «pandas - Python Data Analysis Library». [En línea]. Disponible en: <https://pandas.pydata.org/>
- [16] «Rasterio: access to geospatial raster data — rasterio 1.4.4 documentation». [En línea]. Disponible en: <https://rasterio.readthedocs.io/en/stable/>
- [17] «Desarrollo iterativo e incremental», *Proyectos Ágiles*. [En línea]. Disponible en: <https://proyectosagiles.org/desarrollo-iterativo-incremental/>
- [18] «scikit-learn: machine learning in Python — scikit-learn 1.9.0 documentation». [En línea]. Disponible en: <https://scikit-learn.org/stable/>
- [19] «XGBoost Documentation — xgboost 3.3.0 documentation». [En línea]. Disponible en: <https://xgboost.readthedocs.io/en/stable/>
- [20] «XGBoost», IBM. [En línea]. Disponible en: <https://www.ibm.com/es-es/think/topics/xgboost>

- [21] PyTorch Contributors, «PyTorch documentation». [En línea]. Disponible en: <https://docs.pytorch.org/docs/2.12/index.html>
- [22] «Joblib: running Python functions as pipeline jobs — joblib 1.5.3 documentation». [En línea]. Disponible en: <https://joblib.readthedocs.io/en/stable/>
- [23] «Welcome to the SHAP documentation — SHAP latest documentation». [En línea]. Disponible en: <https://shap.readthedocs.io/en/latest/>
- [24] «Matplotlib — Visualization with Python». [En línea]. Disponible en: <https://matplotlib.org/>
- [25] «seaborn: statistical data visualization — seaborn 0.13.2 documentation». [En línea]. Disponible en: <https://seaborn.pydata.org/>
- [26] Michael Waskom, «seaborn: statistical data visualization», *The Journal of Open Source Software*, vol. 6, n.º 60, p. 3021, abr. 2021.
- [27] «Xarray documentation», Xarray. [En línea]. Disponible en: <https://docs.xarray.dev/en/stable/>
- [28] «dbfread - Read DBF Files with Python — dbfread 2.0.7 documentation». [En línea]. Disponible en: <https://dbfread.readthedocs.io/en/latest/>
- [29] «CoLAB», Google for Developers. [En línea]. Disponible en: <https://developers.google.com/colab>
- [30] «netCDF4 API documentation». [En línea]. Disponible en: <https://unidata.github.io/netcdf4-python/>
- [31] N. Hotz, «What is CRISP DM?», Data Science PM. [En línea]. Disponible en: <https://www.datascience-pm.com/crisp-dm-2/>
- [32] «Ridge Regression», IBM. [En línea]. Disponible en: <https://www.ibm.com/es-es/think/topics/ridge-regression>
- [33] «Welcome to Python.org», Python.org. [En línea]. Disponible en: <https://www.python.org/>
- [34] «Free Citation Generator | APA, MLA, Chicago | Scribbr», Scribbr. [En línea]. Disponible en: <https://www.scribbr.com/citation/generator/>
- [35] Wikipedia contributors, «Google Docs», Wikipedia. [En línea]. Disponible en: https://en.wikipedia.org/wiki/Google_Docs
- [36] «Typst: The new foundation for documents», Typst. [En línea]. Disponible en: <https://typst.app/>
- [37] «Climate Data Store». [En línea]. Disponible en: <https://cds.climate.copernicus.eu/>
- [38] Copernicus Data Space Ecosystem, «Copernicus DEM - Global and European Digital Elevation Model», Copernicus Data Space Ecosystem. [En línea]. Disponible en: <https://dataspace.copernicus.eu/explore-data/data-collections/copernicus-contributing-missions/collections-description/COP-DEM>
- [39] Aurélien Géron, *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow*, 3.^a ed. O'Reilly Media, Inc., 2022.
- [40] Wikipedia contributors, «Arthur Samuel (computer scientist)», Wikipedia. [En línea]. Disponible en: [https://en.wikipedia.org/wiki/Arthur_Samuel_\(computer_scientist\)](https://en.wikipedia.org/wiki/Arthur_Samuel_(computer_scientist))
- [41] A. L. Samuel, «Some Studies in Machine Learning Using the Game of Checkers», *IBM Journal of Research and Development*, vol. 3, n.º 3, pp. 210-229, 1959.
- [42] «LaTeX - A document preparation system». [En línea]. Disponible en: <https://www.latex-project.org/>

- [43] Wikipedia contributors, «Tom M. Mitchell», Wikipedia. [En línea]. Disponible en: https://en.wikipedia.org/wiki/Tom_M._Mitchell
- [44] Reuters, «Computer trained to 'read' mind images of words», Reuters. [En línea]. Disponible en: <https://www.reuters.com/article/us-computer-mind-idUSN2939892820080529/>
- [45] T. M. Mitchell, *Machine Learning*. McGraw-Hill, 1997.
- [46] S. J. Russell y P. Norvig, *Artificial Intelligence: A Modern Approach*, 4.^a ed. Pearson, 2020.
- [47] J. Cohen, «A Coefficient of Agreement for Nominal Scales», *Educational and Psychological Measurement*, vol. 20, n.º 1, pp. 37-46, 1960.
- [48] I. Guyon y A. Elisseeff, «An Introduction to Variable and Feature Selection», *Journal of Machine Learning Research*, vol. 3, pp. 1157-1182, 2003.
- [49] D. Bergmann, «Machine learning». [En línea]. Disponible en: <https://www.ibm.com/es-es/think/topics/machine-learning>

11. Anexos

En los siguientes apartados se introducirán elementos que serán de utilidad par tener un mayor entendimiento sobre las diferentes partes de este TFG.

11.1. Glosario de Términos

En este glosario se incluyen definiciones que puedan ser de utilidad y aclaraciones sobre las diferentes abreviaturas empleadas a lo largo de esta memoria.

Términos	
Abreviaturas/Acrónimos	

Tabla 7: Glosario de términos y acrónimos